VIDEO MANAGEMENT SYSTEM (VMS)
Project Plan & Architecture

Milestone XProtect Style – Complete Feature Breakdown

Generated: 26 August 2026

1. SERVER & CLIENT ARCHITECTURE

Milestone-style VMS follows a distributed client-server architecture. Multiple specialized servers handle different responsibilities for scalability and reliability.

1.1 High-Level Components

Component	Role	Responsibility
Management Server	Central Brain	Camera configuration, Users, Roles, Rules, System settings, License management
Recording Server	Video Engine	Ingest RTSP streams, Record video, Motion detection, Storage management
Event Server	Alarms	Process events, Trigger alarms, Send notifications
Streaming / Media Server	Delivery	Live streaming (HLS/WebRTC), Playback streaming to clients
Database (PostgreSQL)	Storage of Metadata	Camera configs, Users, Events, Bookmarks, Audit logs
Storage (Disk / NAS)	Video Files	Raw video recordings (MP4/MKV), organized by camera + date

1.2 Recommended Server Structure (Folder / Modules)

vms-server/
■■■ management-server/ → Config, Users, API Gateway
■ ■■■ api/ → REST / gRPC endpoints
■ ■■■ auth/ → JWT + Role based access
■ ■■■ camera/ → Camera CRUD + ONVIF discovery
■ ■■■ rules/ → Recording & alarm rules
■■■ recording-server/ → Core video processing
■ ■■■ ingest/ → RTSP client (retina / GStreamer)
■ ■■■ recorder/ → Continuous + Event recording
■ ■■■ motion/ → Motion detection engine
■ ■■■ storage/ → File management + retention
■■■ streaming-server/ → Live & Playback delivery
■ ■■■ live/ → WebRTC / HLS live streams
■ ■■■ playback/ → Timeline based video serving
■■■ event-server/ → Alarms & Notifications
■■■ shared/ → Common models, DB, utils
■■■ config/ → Environment & secrets

1.3 Client Structure

Client Type	Technology Suggestion	Main Features
Web Client	Next.js / React or Leptos	Live view, Playback, Alarms, Camera management, Dashboard
Desktop Client (Smart Client style)	Tauri (Rust) or Electron	Multi-monitor support, Advanced layouts, Keyboard shortcuts, Offline capability
Mobile Client	React Native / Flutter	Live view, Playback, Push notifications, PTZ control
Admin / Management Client	Web based	User management, System configuration, Server status, License

Recommended Communication: Clients talk to Management Server via REST/gRPC. Live video streams come directly from Streaming/Recording servers (WebRTC or HLS) for low latency.

2. PHASED FEATURE BREAKDOWN

Build the system in phases. Each phase delivers a usable product. Do not jump to advanced features before the core is stable.

PHASE 1 – MVP (Minimum Viable Product)

Goal: Working system with Live View + Recording (target: 4–8 cameras)

#	Module	What to Build	Key Details
1	**Camera Management**	Add / Edit / Delete cameras	RTSP URL, Username, Password, Name, Online status check
2	**Live Viewing**	Real-time video display	Single camera + 2×2 / 3×3 grid views
3	**Recording Engine**	Continuous recording	Start/Stop, store as MP4/MKV, date-time folders
4	**Storage Management**	Disk handling	Path configuration, free space monitoring, basic retention
5	**Basic Playback**	Play recorded video	Simple player + date selection + timeline scrub
6	**Web Dashboard**	Main UI	Camera list, Live page, Playback page, basic navigation
7	**Authentication**	Login system	Username + Password, JWT token, single Admin role

Outcome after Phase 1: A working small VMS that can view and record multiple cameras.

PHASE 2 – Core VMS Features

Goal: Real usable VMS with events, multi-view and user control

#	Module	What to Build	Key Details
8	**Motion Detection**	Detect movement	Frame differencing or OpenCV based, sensitivity settings
9	**Event Recording**	Record only on motion	Pre-buffer + Post-buffer, event timeline markers
10	**Advanced Playback**	Professional playback	Multi-camera sync, speed control (0.5x–8x), frame step
11	**Multi-Camera Layouts**	View layouts	1×1, 2×2, 3×3, 4×4, custom layouts, save user layouts
12	**User Management**	Roles & Permissions	Admin / Operator / Viewer roles, camera-level access rights
13	**Camera Groups**	Organize cameras	Groups (Gate, Parking, Floor-1 etc.), group-based views
14	**Search**	Find video	Search by Date + Time + Camera + Event type
15	**Export / Evidence**	Download clips	Select time range → export MP4, optional password protect

Outcome after Phase 2: A solid mid-level VMS suitable for small-medium installations.

PHASE 3 – Professional Features

Goal: Milestone-like professional experience

#	Module	What to Build	Key Details
16	**Alarm System**	Event → Alarm	Motion, Camera offline, Storage full → raise alarm
17	**Notifications**	Alert delivery	Email, Telegram, Browser push, Webhook
18	**Maps**	Visual floor plans	Upload map image, place cameras, click → live view
19	**PTZ Control**	Camera movement	Pan / Tilt / Zoom + presets (ONVIF support)
20	**Bookmarks & Evidence Lock**	Protect important video	Lock clips so retention policy cannot delete them
21	**Audit Logs**	Activity tracking	Who viewed, exported, changed settings – full trail
22	**Multi-Server Support**	Scalability	Multiple Recording Servers under one Management Server
23	**Professional UI/UX**	Polished interface	Dark theme, multi-monitor, keyboard shortcuts, smooth controls

PHASE 4 – Advanced / Enterprise Features

Goal: High-end capabilities

#	Module	What to Build	Key Details
24	**AI Object Detection**	Smart analytics	Person, Vehicle, Bicycle detection (YOLO / ONNX)
25	**Smart Search**	AI-powered search	Search by object type, color, appearance
26	**Failover / Redundancy**	High availability	Recording server failure → automatic takeover
27	**Mobile Application**	Phone access	Live + Playback + Alarms + PTZ on iOS/Android
28	**Cloud / Hybrid Backup**	Offsite storage	Important clips backup to S3 / MinIO compatible storage
29	**Third-party Integrations**	Open platform	Access Control, Alarm panels, REST API for partners
30	**High Scalability**	Large installations	100+ cameras, load balancing, distributed architecture

3. RECOMMENDED BUILD ORDER (Priority)

Start with these 7 modules only (Phase 1):
1. Camera Management → 2. Live Viewing → 3. Recording Engine → 4. Storage → 5. Basic Playback → 6. Web Dashboard → 7. Authentication

After Phase 1 is stable, move to Motion Detection + Event Recording + Multi-layouts + Users (Phase 2).

4. SUGGESTED TECH STACK

Layer	Recommendation	Notes
Camera Ingest	Rust + retina / GStreamer	Best performance & reliability for RTSP
Backend API	Rust (Axum) or Go	Management + Event services
Database	PostgreSQL	Metadata, users, events, config
Video Storage	Local Disk / NAS	Organized by CameraID / Year / Month / Day
Live Streaming	WebRTC or HLS	WebRTC for low latency, HLS for simplicity
Web Client	Next.js + TypeScript	Fast development + good ecosystem

Layer	Recommendation	Notes
Desktop Client	Tauri (Rust)	Lightweight, native feel, multi-monitor
AI Detection	ONNX Runtime + YOLO	Optional in Phase 4

This document is a practical development roadmap for building a Milestone-style VMS. Start small (Phase 1), make it stable, then expand. A full enterprise VMS is a multi-month / multi-year effort even with AI assistance.

Document generated for project planning purposes.